SCE 672: Advanced Robotics

Assessment 2 Presentation: Flatness-Based Chase of AprilTag 3

Student: Ameen Ur Rehman (202527430)
Instructor: Prof. Aamer Iqbal Bhatti
Code: `https://github.com/AmeenUrRehman/RoboMaster-Advanced-Robotics`
Folder: `Assignment 2`

In one sentence: a RoboMaster EP finds AprilTag 3, turns to face it, drives a smooth planned path to it, and stops about 20 cm in front of it. This document follows the five presentation points in order: (1) the planned path in the GitHub code, (2) where the command is sent to the robot, (3) the reference graphs of velocity and x, (4) the tracking errors, and (5) the video. All numbers come from the code and from the final run's log (`logs/flat_run5/motor_voltage_log.csv`).

Words to know

Word	Plain meaning
RoboMaster EP	The small DJI robot with four mecanum wheels and a camera.
AprilTag	A printed black-and-white square, like a simple QR code. The robot follows **Tag 3**; Tags 1 and 2 are decoys.
ROS 2	The software "post office": programs send each other messages on named channels called *topics*.
`cmd_vel`	The topic that carries the drive order to the robot: forward speed v and turn rate ω.
v	Forward speed, in metres per second (m/s).
ω **(omega)**	Turning speed, in radians per second (rad/s). $\omega = 0$ means drive straight.
x	How far the robot has driven forward since the chase started (m).
d_0	Distance from the robot to the tag when the chase starts: 1.053 m in the real run.
T	Time the plan allows for the whole chase: 8.53 s in the real run.
Reference	What the plan says the robot *should* do at each moment.
Tracking error	Where the robot really is minus where the plan says it should be.
Differential flatness	If we plan only the position (x, y) over time, the speed v and turn rate ω follow from a simple formula. No heavy optimisation is needed.
Cubic polynomial	A smooth curve $a_0 + a_1 t + a_2 t^2 + a_3 t^3$: it starts and ends gently, with no jerks.
Standoff	The gap left in front of the tag at the end: 0.20 m.

The big picture

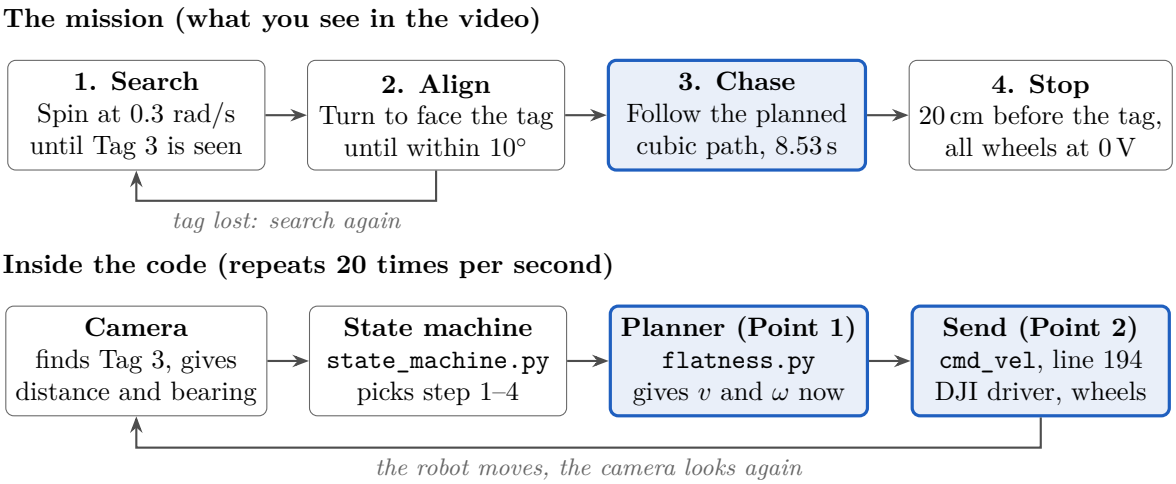


Figure 1: Top: the four steps of the mission. Bottom: the loop the code runs every 0.05 s. The blue boxes are presentation Points 1 and 2.

The four steps in detail: when does the robot switch?

The file `state_machine.py` decides which step the robot is in. Each arrow below is a rule in that file, with its real number.

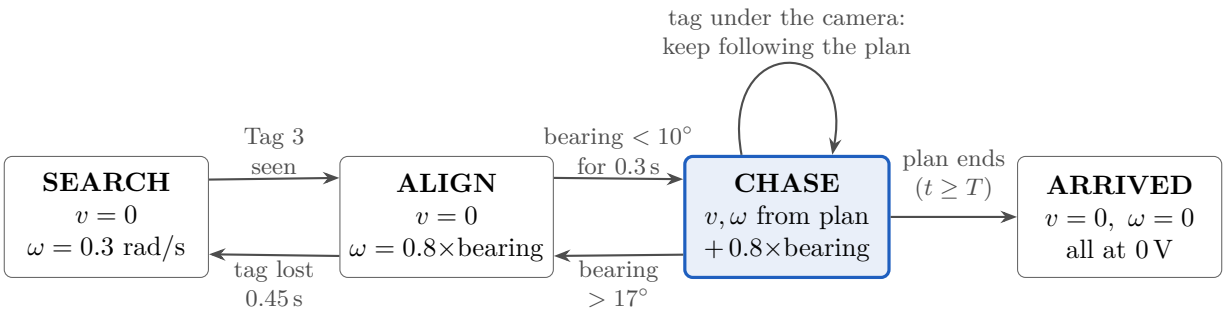


Figure 2: The state machine. "Bearing" is the angle between the camera's centre line and the tag. Once chasing, the robot keeps following the plan even when the floor tag goes under the camera (tag last seen within 1.8 m). The code also stops at once if the camera or the ToF sensor reads 0.20 m or less; in the final run the stop came from the plan ending.

Point 1: How the path is planned (GitHub code)

The path is a smooth straight line to 20 cm in front of the tag. It is planned once, at the moment the chase starts. File: `Assignment 2/rm_flatness_nav/rm_flatness_nav/flatness.py`.

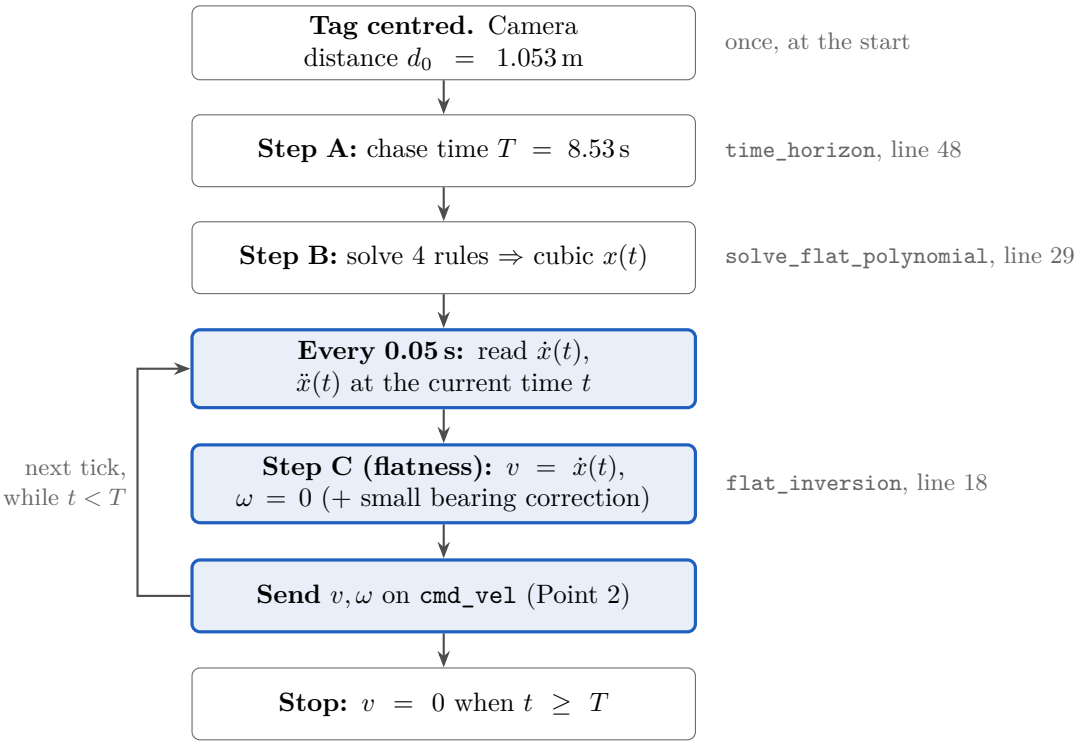


Figure 3: The planner at a glance. White boxes run once when the chase starts; blue boxes repeat 20 times per second until the plan ends.

Step A: How long should the chase take? (`time_horizon`, line 48)

$$T = \max\left(4.0,\ \frac{1.6\,(d_0 - 0.20)}{v_{\max}}\right), \qquad v_{\max} = 0.16\ \mathrm{m/s}.$$

Real run: $d_0 = 1.053\,\mathrm{m}$, so $T = 1.6 \times 0.853/0.16 = \mathbf{8.53\,s}$. A farther tag gets more time, so the top speed stays about $0.137\,\mathrm{m/s}$.

Step B: Draw a smooth curve for $x(t)$ (`solve_flat_polynomial`, line 29)

The curve $x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$ must obey four rules:

Rule	Meaning
$x(0) = 0$	start where the robot is now
$\dot{x}(0) = 0.06$ m/s	start with a small speed
$x(T) = d_0 - 0.20 = 0.853$ m	end 20 cm before the tag
$\dot{x}(T) = 0$	arrive with zero speed

Four rules and four unknowns: the code solves them with `numpy.linalg.solve`. Result for the real run:

$$x(t) = 0.06\,t + 0.0211\,t^2 - 0.00192\,t^3.$$

The sideways curve $y(t)$ is zero, because the robot first turns to face the tag.

Step C: Turn the curve into driving orders (flatness) (`flat_inversion`, line 18)

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \omega = \frac{\dot{x}\,\ddot{y} - \dot{y}\,\ddot{x}}{\dot{x}^2 + \dot{y}^2}.$$

With $y = 0$ this gives $v = \dot{x}(t)$ (the slope of the curve) and $\omega = 0$ (drive straight). No optimiser is used, only this formula.

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):     # flatness.py, line 29
    A = np.array([[1.0, 0.0, 0.0,    0.0],
                  [0.0, 1.0, 0.0,    0.0],
                  [1.0, T,   T**2,   T**3],
                  [0.0, 1.0, 2.0*T,  3.0*T**2]])
    b = np.array([z0, zdot0, zf, zdotf], dtype=float)
    return np.linalg.solve(A, b)                         # [a0, a1, a2, a3]
```

Point 2: Where the command is sent to the RoboMaster

The command leaves the code in one place: `nav_node.py`, function `tick()`, which publishes on the `cmd_vel` topic 20 times per second.

```
# nav_node.py, line 103: create the "mailbox" to the robot
self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', 10)

# nav_node.py, lines 190-194: runs every 0.05 s (20 Hz)
v, w = self.fsm.update(obs)                 # the plan gives speed v and turn w
tw = Twist()
tw.linear.x, tw.angular.z = float(v), float(w)
self.cmd_pub.publish(tw)                    # <- sent to the RoboMaster here
```

The chain, step by step:

1. `tick()` runs 20 times per second (`control_rate = 20`).
2. It asks the state machine for v and ω. During the chase this is `_chase()` in `state_machine.py` (line 131), which reads the planned curve at the current time with `self.traj.sample(t)`.
3. It packs v and ω into a `Twist` message and publishes it on `cmd_vel`.
4. DJI's `robomaster_ros` driver (started with `ros2 launch robomaster_ros ep.launch conn_type:=rndis`) listens on `cmd_vel` and sends the speeds over the USB cable.
5. The robot's own controller turns the four wheels.

Safety: when the plan ends, or Ctrl+C is pressed, the node sends zero speed (`shutdown()`, line 225), so the robot always stops.

Point 3: Reference graphs of velocity and x

The reference starts at $0.06\,\mathrm{m/s}$, peaks at $0.137\,\mathrm{m/s}$ in the middle, and reaches zero exactly when x reaches $0.853\,\mathrm{m}$ at $T = 8.53\,\mathrm{s}$.

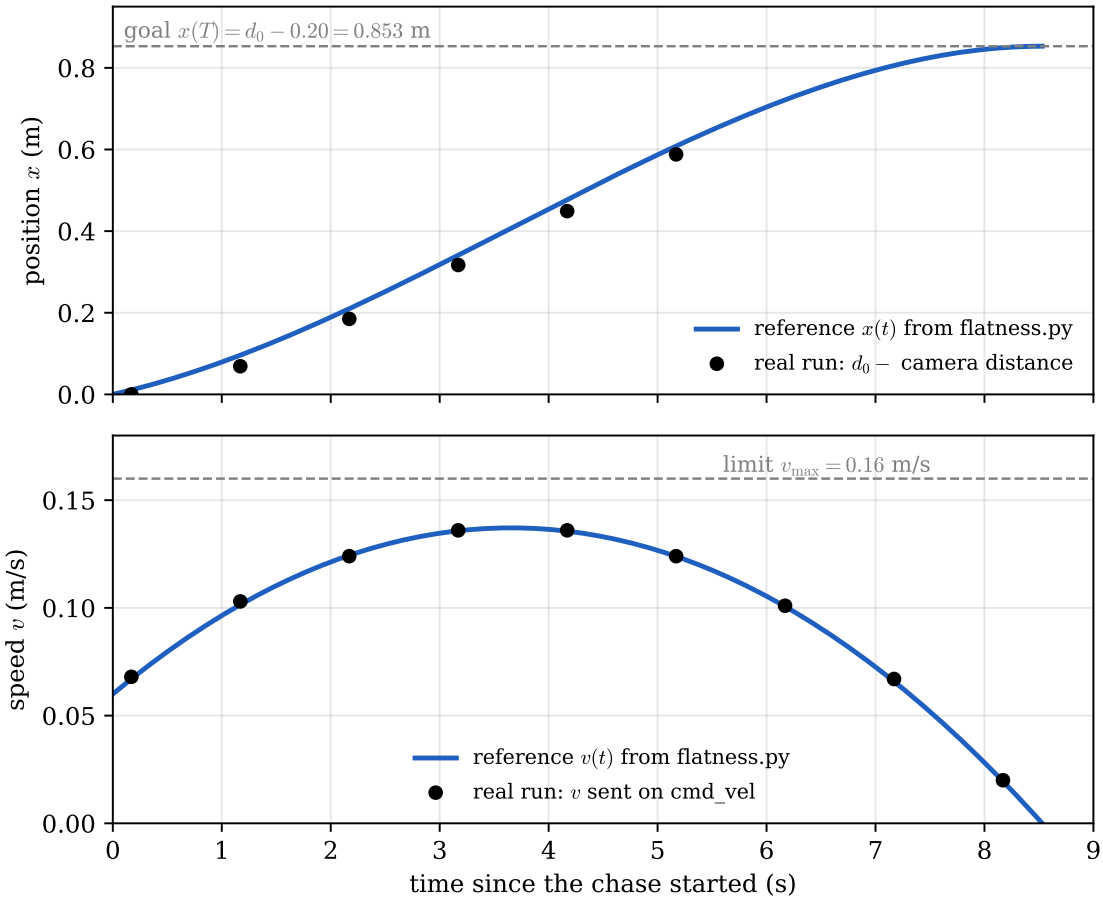


Figure 4: Blue lines: the reference computed by `flatness.py` for $d_0 = 1.053$ m. Dots: the real run (1 sample per second). Top: position; the dots are d_0 minus the camera distance to the tag. Bottom: speed; the dots are the values sent on `cmd_vel`. Time zero is the start of the chase (9.83 s in the log).

How to read it. The top curve is S-shaped: slow start, fast middle, gentle stop. The bottom curve is its slope (the speed): a hill that never reaches the dashed 0.16 m/s limit.

Point 4: Tracking errors

The robot stayed within **3 cm** of the planned position and within **4°** of the tag's direction. It ran slightly behind the plan.

How it is measured. The camera measures the distance to the tag in every frame, so

$$x_{\text{real}} = d_0 - \text{camera distance}, \qquad \text{tracking error} = x_{\text{real}} - x_{\text{plan}}.$$

Time in log (s)	Planned x (m)	Real x (m)	Position error (cm)	Heading error (deg)
10	0.011	0.000	−1.1	3.9
11	0.096	0.069	−2.7	1.0
12	0.210	0.185	−2.5	0.2
13	0.341	0.317	−2.4	0.3
14	0.478	0.449	−2.9	0.7
15	0.608	0.588	−2.0	0.4

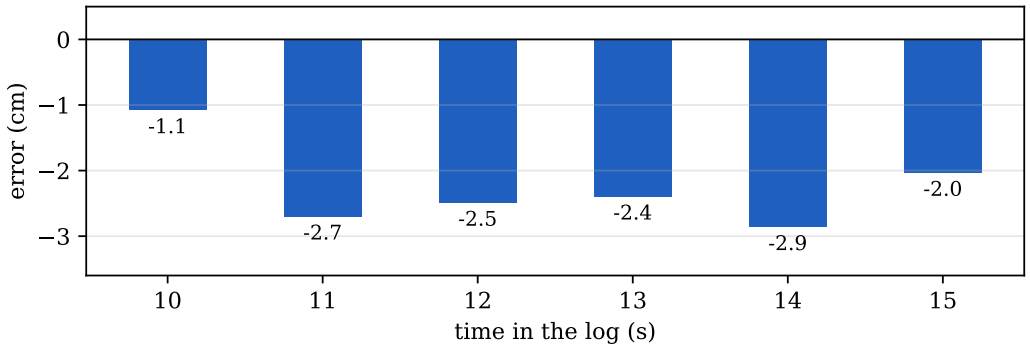


Figure 5: Position tracking error at each logged second. Minus means the robot was slightly behind the plan.

End point: planned gap 20 cm, real gap about 20 cm (measured by hand).

Why the error is small. The robot first turns to face the tag, so the path is a straight line. While the tag is visible, a small correction (0.8× bearing) is added to ω.

Why the robot still stops at about 20 cm

- **The error is small next to the path.** The robot was 1 to 3 cm behind the plan on a path of 85 cm, about 2 to 3%. A gap of "about 20 cm" measured by hand cannot separate 20 from 22 cm.
- **The robot finishes the plan by itself.** After 15 s in the log the floor tag is too close to see (it is under the camera). The code keeps sending the planned speeds until $T = 8.53$ s is over, then sends zero. So the robot drives the whole planned 0.853 m.
- **Part of the "error" may come from the camera.** The camera does not measure distance directly. It computes it from the tag's size in the image and from the camera's downward tilt (about 20°, read from the images). A small error in the tag size or the tilt shifts every distance by a centimetre or two, which looks like a lag. The robot may therefore have been even closer to the plan than the table shows.

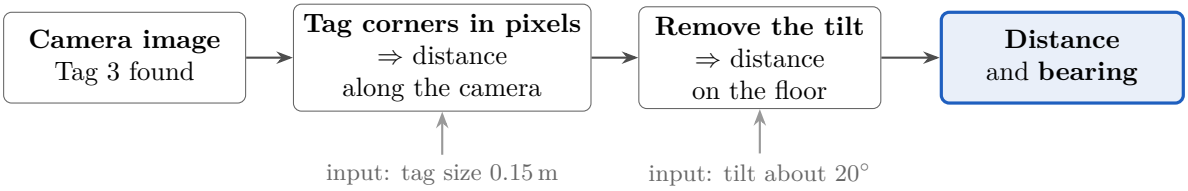


Figure 6: How the camera turns an image into a distance (`tag_detector.py`). The two grey inputs are set by hand, so a small error in either shifts the measured distance.

Point 5: Video

The demonstration videos (each 45 s or less) show the four steps of the mission.

- Video 1: `https://youtu.be/EaZKQfd3usE`
- Video 2: `https://youtu.be/JlHnuVnLLrA`

1. **Search**: the robot turns on the spot until it sees Tag 3.
2. **Align**: it turns to face the tag.
3. **Chase**: it follows the planned path: slow start, faster middle, slow end.
4. **Stop**: it stops about 20 cm before the tag with all wheel voltages at 0 V.